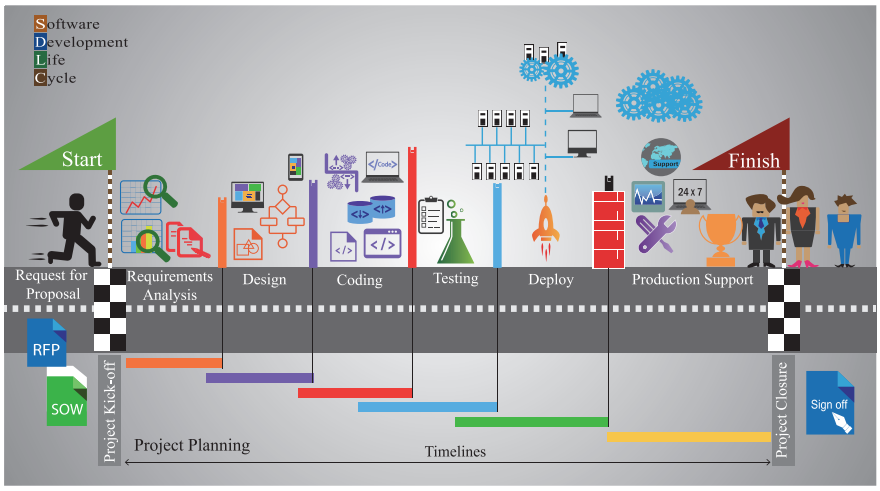
The Software Development Lifecycle

BRINGING a computer programming project to market is a time consuming and expensive effort. The key to completing it on time, within budget, and ensuring that it meets user requirements requires planning. The Software Development Lifecycle describes the steps in this process, and programmers must be conversant with this process.



Objective:

Describe the planning, analysis, and the design phase of an application.

Key Terms:

- Adaptive Model
- Agile manifesto
- Agile Model
- Back-end coding
- Content Management System (CMS)
- Daily scrum meeting
- Development Environment
- Front-end coding
- Integration Testing
- Key Performance Indicators (KPIs)
- Predictive model
- Product backlog
- Production Environment
- Quality Assurance environment
- SDLC (Software Development Lifecycle)
- Sprints
- Subject Matter Experts (SMEs)
- System Testing
- The Scrum Model
- User Acceptance testing
- Unit Testing
- User Interface (UI)
- Waterfall Model

Copyright © by CAERT, Inc. — Reproduction by subscription only.

The Software Development Lifecycle

Bringing a computer programming project to market is a time consuming and expensive effort. The key to completing it on time, within budget, and ensuring that it meets user requirements, requires planning. The Software Development Lifecycle describes the steps in this process, and programmers must be conversant with this process.

DESCRIBE THE PLANNING, ANALYSIS, AND THE DESIGN PHASE OF AN APPLICATION

In this section, you will look into the steps involved in planning, analyzing and designing a software application.

SDLC (Software Development Lifecycle)

Software development is a time-consuming effort. It can take months, and even years, to develop an application. AAA games typically take 1 to 2 years to bring a game to the market, while mobile apps take 3 to 5 months to develop. The steps in getting an app to the market are described by the **SDLC (Software Development Lifecycle)**. The SDLC breaks down the process to bring an application to market into 7 phases. The purpose of phase in the SDLC can encapsulated in a phrase which indicates the basic rationale of that phase. The software development process starts when a customer requests an application, or a perceived need is detected in the marketplace and ends when the app

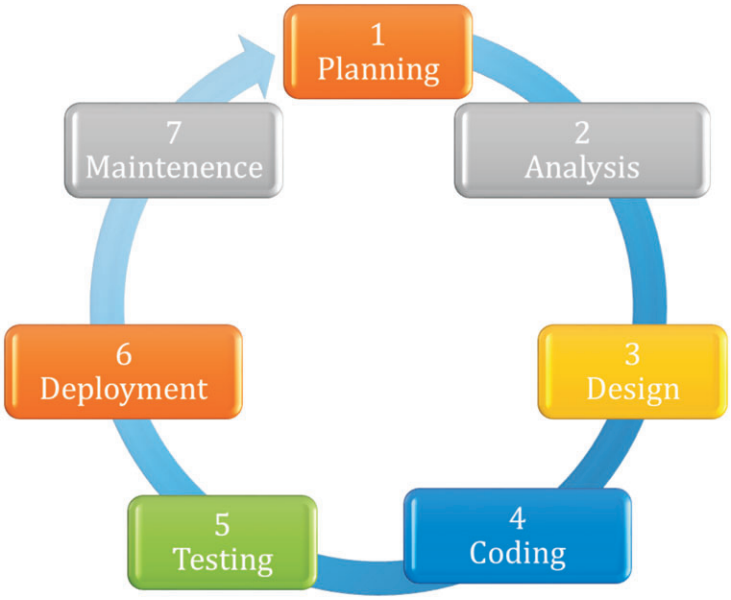


FIGURE 1. The SDLC breaks down the process to bring an application to market into 7 phases.

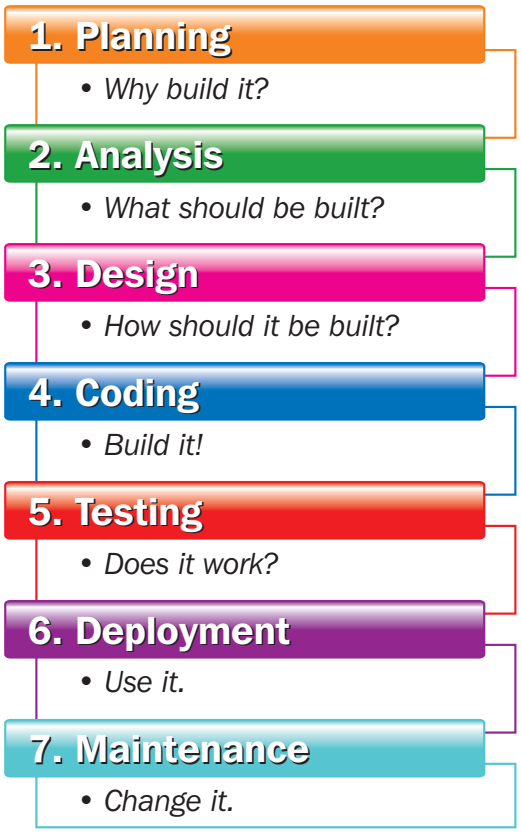


FIGURE 2. The purpose of each phase in the SDLC.

Copyright © by CAERT, Inc. — Reproduction by subscription only.

has been deployed. The first, second and third phases in the SDLC are the Planning, Analysis and Design Phases.

Planning Phase

The Planning phase decides whether the app should be developed or not. Various stakeholders in the project are assembled and the reasons for developing the app are discussed. This phase examines the business aspects of the proposed app and determines scope of the project, along with preliminary cost estimates. Feasibility studies are also conducted, and a tentative timeline is proposed.

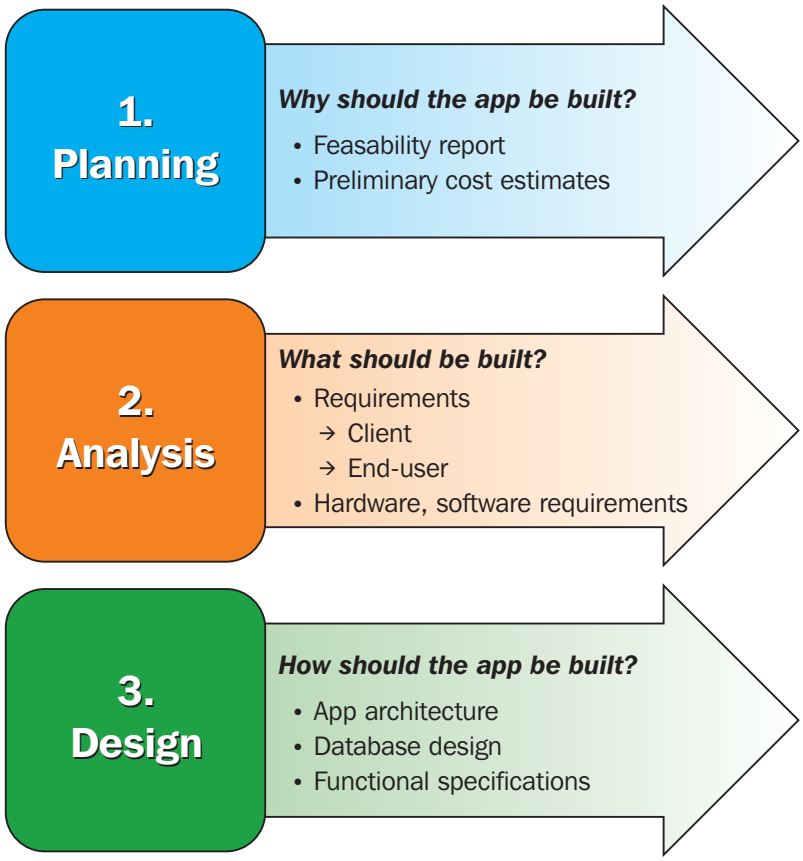


FIGURE 3. The first 3 phases of the SDLC.

Analysis Phase

The Analysis phase is commenced after the decision to develop the app has been made. In this phase, the requirements of the proposed app are finalized. This phase is performed by members of the development team, business analysts, **Subject Matter Experts (SMEs)**, customers and end-users who will ultimately use the app. The SMEs provide subject matter expertise about the app requirements, and the marketplace for the app. The next phase, the Design phase, depends upon the accuracy of the requirements gathered in this phase, which ends with the creation of a high-level design document.

Design Phase

Phase 3 of the SDLC is the Design phase. In this phase, the team decides upon the building blocks that will be used to build the app. The architecture of the application is decided. This includes decisions about the application frameworks, database software, and development of function specifications. For a web app, this includes web site design, and for a mobile app, it includes screen layouts. Server configurations are also finalized. Decisions about programming language may be made in the Design phase or deferred until the Coding phase.

Case Study

To understand the SDLC better, a case study approach is used to demonstrate the use of the SDLC. Consider a local pizzeria that needs to build a web app that allows its customers to

Copyright © by CAERT, Inc. — Reproduction by subscription only.

place orders for various types of pizzas that the pizzeria makes via a website. The customer must be allowed to make payments via the web app and specify the time that they will pick up the pizza.



FIGURE 4. Case study example.

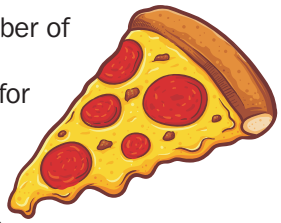
Planning Phase

The requirements from the customer are stated in business terms. The Pizzeria's offerings (pizzas) and their current mode of doing business must be understood, along with other information. If feasibility studies and costing estimates have indicated that building the app is a sound business decision, the project moves ahead.

Analysis Phase

In this phase, there are 3 sets of people involved. The first are the members of the design team. They bring in technical and software expertise. The customer is the pizzeria, while the pizzeria's customers are those who place orders for their product. These are the end-users of the app. It is the responsibility of both the design team and the pizzeria team to correctly understand and anticipate the requirements of the end-users, the pizzeria's customers. Using focus groups of potential end-users will help refine requirements. Many questions must be answered in the Analysis phase. These questions will need be viewed by both the client and the end-user. Answers to these questions impact the design of the application.

Here are some of the questions to be answered:



- What is the minimum and maximum number of pizzas that can be ordered?
- Can a different pick-up time be specified for each pizza?
- Can different types of pizzas be ordered?
- Should payment be made separately for each pizza, or can payment be done once for all pizzas ordered?
- Can a bill be split between multiple credit cards?
- Is there a discount if a certain number of pizzas are ordered?
- How will discount coupons (if they exist) be handled?
- How will gift-cards (if they exist) he handled?
- Will this be a "Groupon" site?

FIGURE 5. Customer requirement — Details. Many questions must be answered in the Analysis phase. These questions will need be viewed by both the client and the end-user. Answers to these questions impact the design of the application.

Design Phase

The Design phase commences after the Analysis phase has con-

Copyright © by CAERT, Inc. — Reproduction by subscription only.

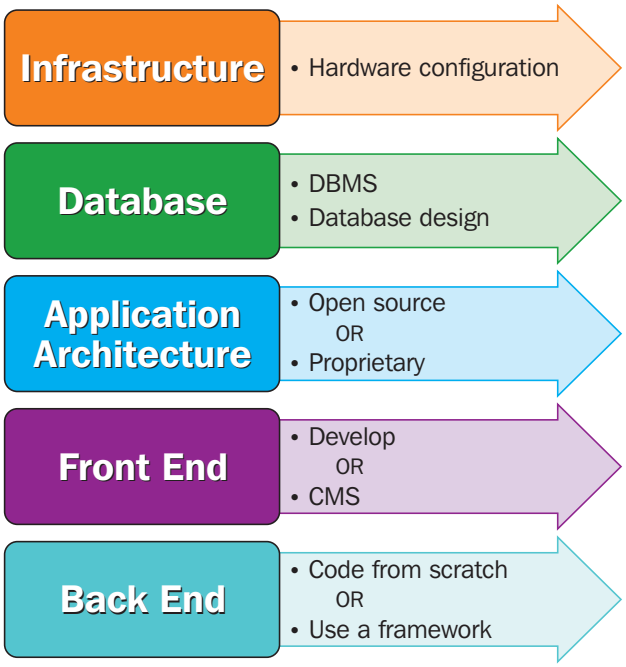


FIGURE 6. High level Design Document. Items to be finalized in the Design phase.

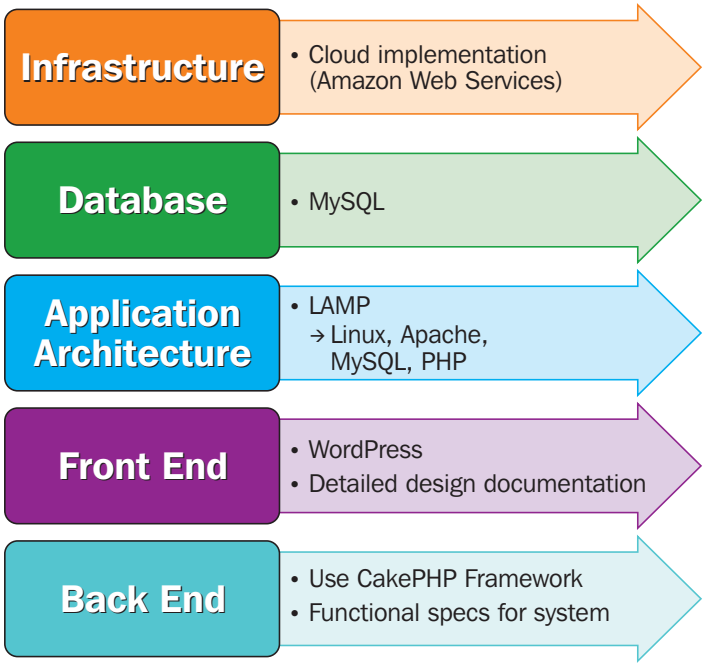


FIGURE 7. Completed High-level Design Document.

cluded. The Design document contains various information items along with clarifying details. The Database Management Software is selected, along with a design of the database to hold information. Application architecture is finalized. This includes determining if cloud architecture will be used or whether the Pizzeria will maintain hardware. Decisions to create the front-end from scratch or use a Content Management system is made, and frameworks are selected for the back end. In addition to this high-level view, detailed documents are created showing screen-shots of proposed front-end views and functional specifications for front-end logic and back-end programs. These functional specifications are implemented in the Coding phase.



DIGGING DEEPER...

UNCOVERING ADDITIONAL FACTS: What Does SDLC Stand for?

This lesson discusses the Software Development Lifecycle, abbreviated to SDLC. However, there is another process that is also abbreviated to SDLC, and it is the System Development Lifecycle. The System Development Lifecycle is used as a tool to plan IT projects, and it includes hardware and software. It is considered a superset of the Software Development Lifecycle.

To learn more about the System Development Lifecycle, and to understand is differences from the Software Development Lifecycle, go to https://securityintelligence.com/the-system-development-life-cycle-a-phased-approach-to-application-security/.

Copyright © by CAERT, Inc. — Reproduction by subscription only.

USE THE DESIGN TO DELINEATE TEAM DEVELOPMENT ASSIGNMENTS

In this section you will learn about the various SDLC implementations and understand how each implementation allocates software development tasks amongst team members.

SDLC Implementations

The SDLC is a high-level model that depicts all the phases in a software development effort. There are many implementations of the SDLC model. Though the steps in the SDLC model are universal, they are implemented differently in each of the SDLC variants. SDLC Models can be classified by categories. These categories are Predictive Models, Adaptive Models and Agile Models.

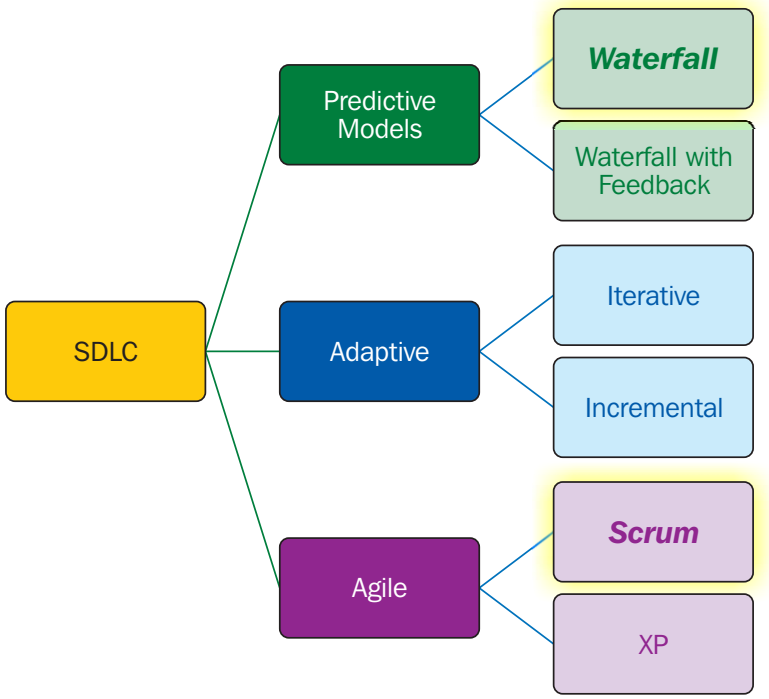


FIGURE 8. Categories of SDLC Implementations.

Predictive Models

The oldest of these models is the **Predictive model**. In this model, the SDLC phases are completed one after the other, with no backward movement. For example, the Design Phase is completed before the Coding Phase is stared. If there needs to be a change in the app's design, it cannot be incorporated once the Coding phase has commenced. This can lead to the development of apps that do not meet customer or end-user needs.

The Waterfall Model: The most popular Predictive model, and one the earliest SDLC implementations, is the **Waterfall model**. It is a sequential model where a phase is started only after the previous phase has been completed. In this model, it is critical that requirements be correctly and accurately identified in the Analysis phase. Once the Design phase starts, modifications cannot be incorporated since the Waterfall model does not permit backward phase movements. If a project's requirements are well defined, and the scope of the project is small, the Waterfall model can be used to successfully. System design and Development is relatively easy, and the scope and cost of the project can be predicted early in the process. However, if the project requirements are unclear and evolving, the Waterfall model's shortcomings

Copyright © by CAERT, Inc. — Reproduction by subscription only.

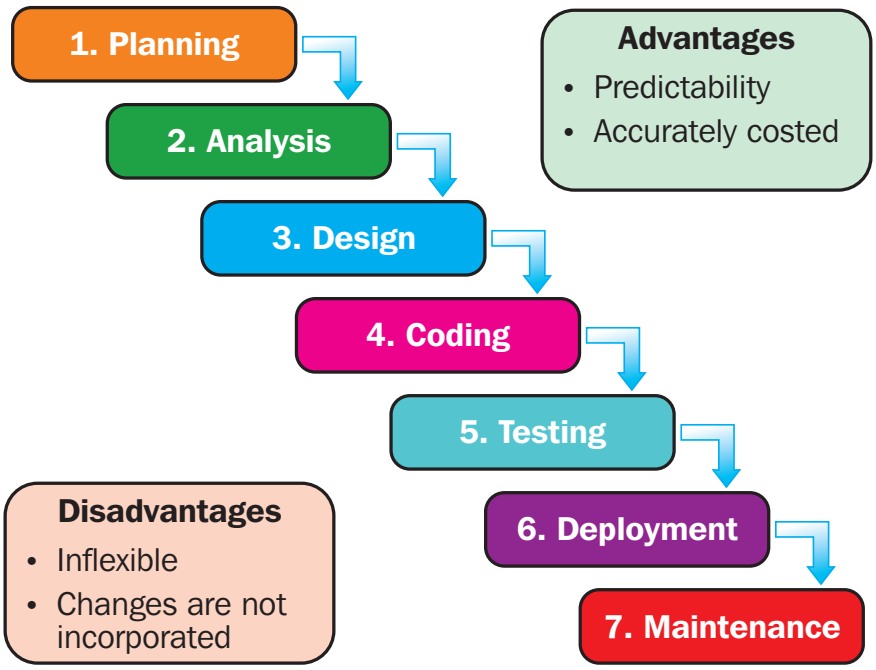


FIGURE 9. The SDLC Waterfall Model.

become very evident. It does not allow for an iterative approach where changes are incorporated continuously throughout the lifetime of the development effort.

Adaptive Models

The second SDLC category is the **Adaptive Model**. This is also referred to as the Iterative Model. Here, the app is built incrementally. The first build creates a small section of the software that is functional and useful. If it does not meet customer requirements, it is modified, and the process is repeated till it is finalized, making this an iterative process. Once a section has been completed, the next increment is developed. This model allows for changes to be made during the Coding phase, and this leads to applications that meet customer need more closely than those built using predictive models.

Agile Model

The third implementation of the SDLC is the **Agile model**. In 2001, seventeen software developers met in Snowbird, Utah, and developed the Manifesto for Agile Software Development, which is commonly referred to as the **Agile Manifesto**. This document did not contain detailed implementation plans but placed emphasis on an iterative approach to software development. The premise was that while tools and processes are important, people and interactions are more important. It incorporates elements of the Adaptive model with emphasis on speed of delivery of software. Agile projects are implemented in small increments by compact self-managed teams. The Agile model does not believe in designing the entire app before starting the Coding phase. Instead, an incremental and iterative approach is used. The project is broken into smaller pieces and each of these is competed in small sections of time. The cus-

Copyright © by CAERT, Inc. — Reproduction by subscription only.

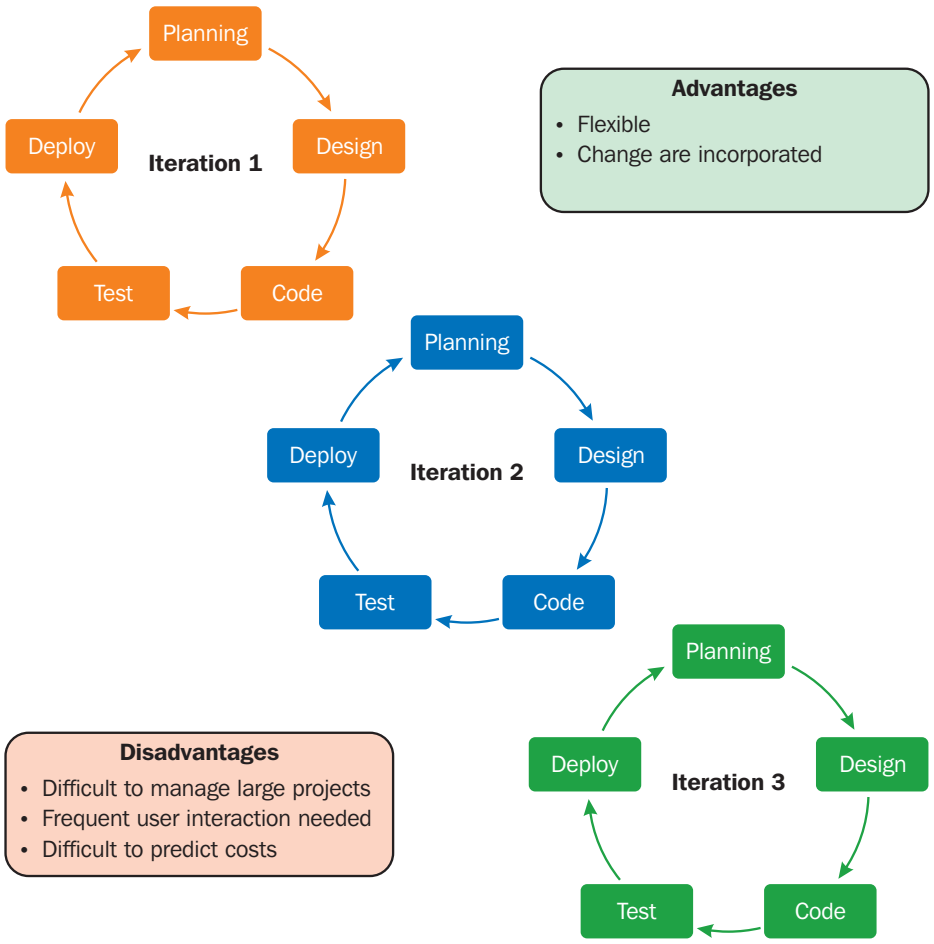


FIGURE 10. The Agile Model.

tomer is part of the development process and provides feedback each step of the way, allowing for changes to be incorporated as the app is being built. In Agile, multiple iterations are performed. An iteration can be a re-working of existing code that needs to be changed due to changes in user requirements, or it can mean the development of new software modules. Each iteration runs for a few weeks. However, the Agile approach has its drawbacks. Users must commit time towards the development process, and it can be difficult to predict costs for the project.

The Scrum Model. One of the most popular variations of the Agile methodology is the **Scrum Model**. Scrum emphasizes simplicity and frequent interactions between developers and customers. There are 3 main roles in a Scrum project. The Product Owner is the interface between the customer and the development team. This person assembles customer requirements, referred to as stories, and places them in the **product backlog**. The Scrum Master is the person who ensures that the development team can work without any obstacles. Scrum teams are self-organizing, and do not need a formal team leader or project manager. Team members self-select roles during the Coding phase based upon their competencies. The success of the Scrum methodology rests upon the fact that team members feel empowered to per-

Copyright © by CAERT, Inc. — Reproduction by subscription only.

form tasks that fits their skill set best. Scrum projects are broken into smaller timeboxed events called **sprints**. A sprint is a development effort that lasts 3-4 weeks. A sprint starts with a sprint planning meeting. At this meeting, sprints are removed from the product backlog and placed in the sprint backlog. The Development team begins coding and testing the assigned sprint. **Daily scrum meetings** are held at the start of business day. The meeting is a standing-only meeting which lasts 10-15 minutes, where the team discusses their progress during the previous day. Once a sprint is completed, it is reviewed and presented to the Product Owner. If the Product owner accepts the sprint, the sprint is ended, and it is moved into production. A sprint retrospective is also performed to review the sprint completion process to see if any lessons can be learned from the process.

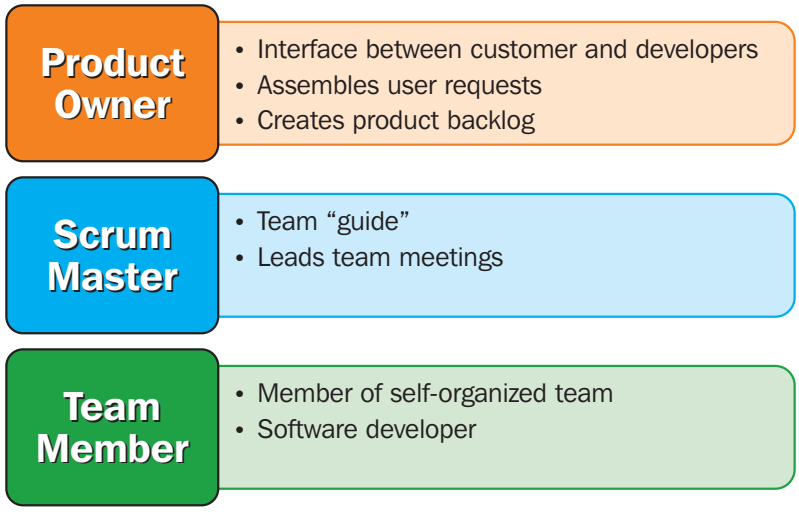


FIGURE 11. Roles in a Scrum project.

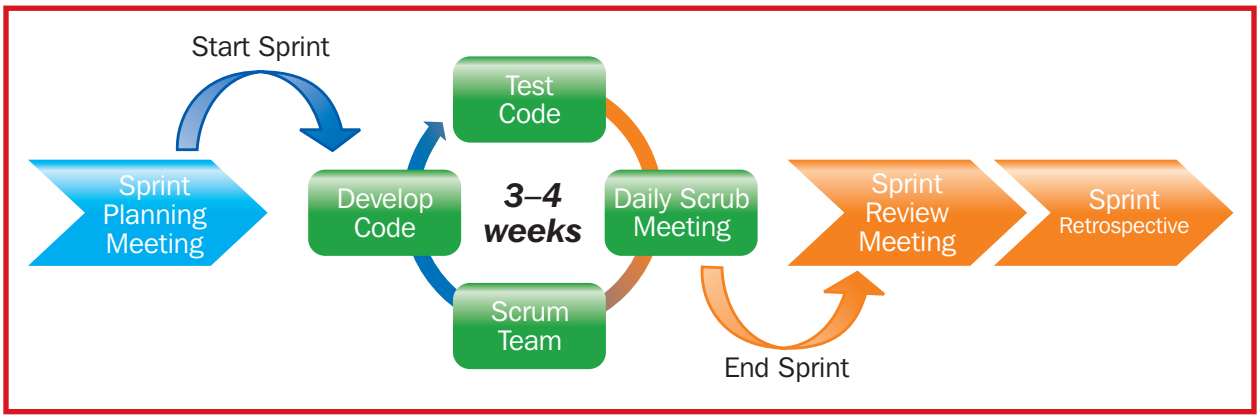


FIGURE 12. Steps in a Sprint.

Determine Team Assignments

Software development is seldom a one-person effort; it is performed by teams of programmers. In a predictive model, an experienced programmer takes up the position of team leader and assembles a team based upon project requirements and skill set of available programmers. In a Scrum project, there is no team leader. The Scrum Master ensures that the team has access to required resources and resolves obstacles that impede project progress. Scrum teams are self-forming, with team members creating their own teams based upon their skills.

Copyright © by CAERT, Inc. — Reproduction by subscription only.

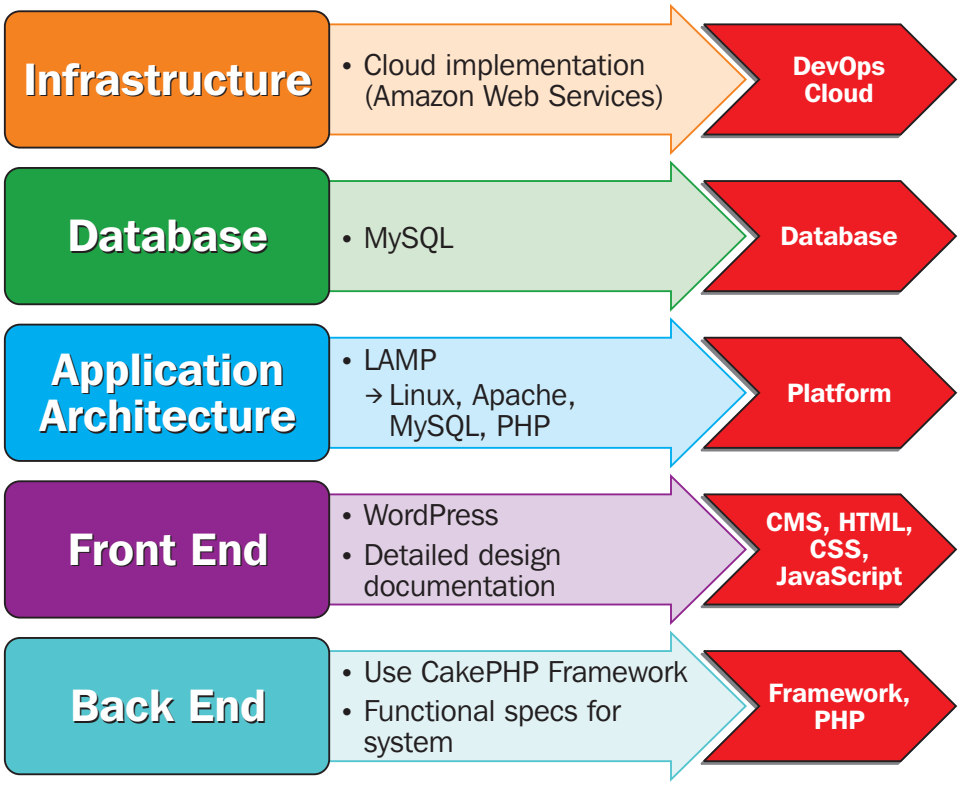


FIGURE 13. Design phase decisions.

Case Study

The Pizzeria client's Design document has been finalized. It details the Design decisions and the skill set needed to complete the tasks in each section of the Design phase.

SELECT THE TYPE OF CODING

In this section you will learn the steps in implementing the Design phase requirements into software code.

Coding Phase

The Coding phase starts when a Design document has been finalized. In a Waterfall project, this phase starts when the Design phase has concluded. In a Scrum project, the Analysis, Design and the Coding phases run concurrently. There are various categories of activities that are completed during the Coding

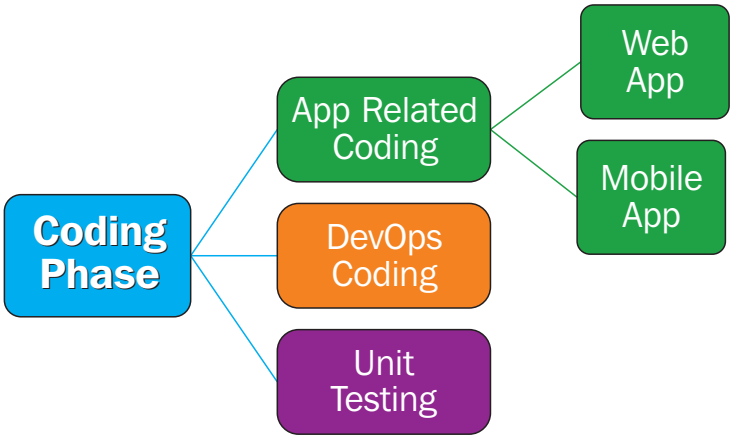


FIGURE 14. Coding Phase Activities.

Copyright © by CAERT, Inc. — Reproduction by subscription only.

Phase. The App related coding step creates code to meet user requirements. The DevOps category includes setting up infrastructure and using tools to speed up coding. The Unit testing step tests code that has been written to ensure that it meets design specifications.

App Related Coding

The Design document decides whether the app should be built as a web-app or a mobile app. The steps to complete a web app are different from those to complete a Mobile app. Keep in mind that web apps may also be accessed from a Mobile device via the mobile device's web browser software app.

- Web app: Coding for a web app is broken into 2 categories — Front-end coding and Back-end coding. Different sets of skills are needed for each of these activities. **Front-end coding** refers to the coding that creates the **User Interface (UI)**. This refers to creating the website layout and designing individual web pages. This requires graphic design skills because usability of a website depends upon color choices, page layout, and font and image choices. Front-end code executes on the PC within the browser software window. Webpages are built using HTML, a scripting language and CSS, which is a technology to format HTML. Depending upon the preferences of the customer and the development team, code can be created from scratch, or a **Content Management System (CMS)** can be used. A Content Management System is a software app that provides partially pre-built content which can be customized. Examples of CMS include WordPress and Drupal. HTML and CSS create static pages. Interactivity is built into a webpage using JavaScript code. Therefore, Front-end coding requires JavaScript coding to be included into webpages. CMSs may provide JavaScript as part of pre-built code and it can be modified. **Back-end coding** refers to code that executes on the webserver. Common languages used in the back-end are Java and PHP. Frameworks are used at the back-end to speed up code production. A backend framework is conceptually like the CMS at the front-end. It provides pre-built code that can be used directly or modified. Common PHP frameworks include Laravel and CakePHP, while common Java frameworks include JavaServer Faces and Struts. A framework is chosen after the coding language has been selected since a framework provides pre-built code in a programming language.

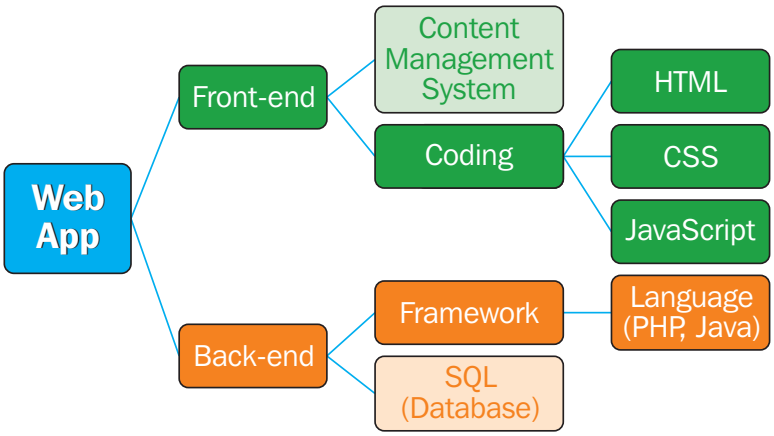


FIGURE 15. Categories of Coding in a Web App.

- Mobile app: Mobile apps run natively on a mobile device. The most popular language used to code apps for an Android is Java. The software used to develop Android Java apps is called Android Studio. Mobile Android Apps are created using Java code along with XML files. Mac mobile devices are developed in a language called Swift. The Xcode IDE is used to create Mac platform apps. Mobile apps can also have a back-end piece if the app

Copyright © by CAERT, Inc. — Reproduction by subscription only.

wishes to connect to a webserver to store data in a database and make use of a web server's processing power. The back-end portion coding considerations are the same as those for a web app.

DevOps Coding

Configuring servers or creating cloud entities can be done manually, but it is more efficient to create scripts to complete tasks. Traditionally scripting for Unix or Linux servers is done using shell scripting languages, such as Bash shell. Servers in a Windows environment use PowerShell to create scripts. Python is rising in popularity for creating scripts used by DevOps in the Unix world. Python is a full-fledged programming language and is more difficult to learn than Bash shell but is also more powerful.

Unit Testing

Programs are tested once they are completed. This process is known as **Unit Testing**. Developers create Test Data cases which demonstrate that the program works as required and as expected. During the Coding phase, apps are tested in a sandboxed environment called the **Development Environment**. This is a non-stable environment where code is constantly modified as it is modified and finalized. When all code has been unit tested, the Coding phase is deemed complete and code is passed on to the Testing Phase for further testing.

Testing Phase

After code has been unit tested, it must be retested with other programs in the application to ensure that programs work well with one another. There are multiple levels of testing before an application can be deployed.

Integration Testing

Programs that belong in the same module are tested together to see if outputs from 1 program flow smoothly into the next. In our Pizzeria case study, all front-end programs that display Pizza options and time delivery screens can be placed in a pool for integration testing. Back-end programs that accept payment options from the user and produce a receipt can be part of another pool to be tested together. In this step, programs are tested in an environment called the **Quality Assurance environment**. Unlike the Coding environment, this is a stable environment and it mimics the final environment that the application will run in. In this phase, QA personnel test the program. DevOps procedures automate test runs using tools and scripts to

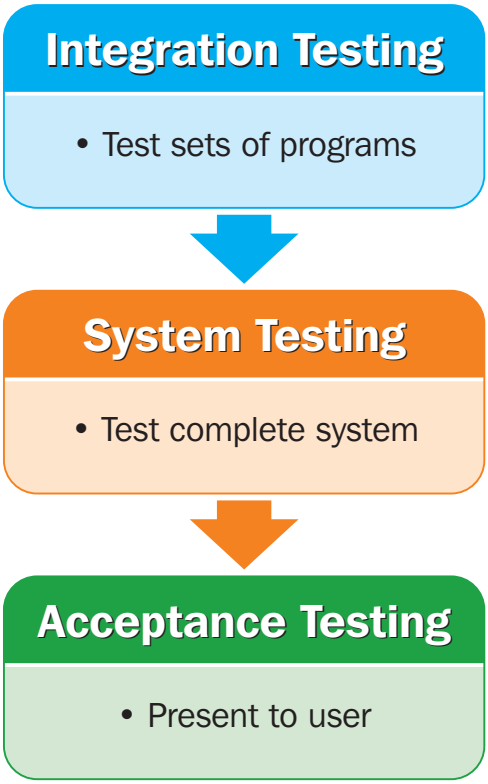


FIGURE 16. Categories of Testing conducted in the Testing Phase.

Copyright © by CAERT, Inc. — Reproduction by subscription only.

speed up this phase. Rigorous test data cases are compiled, and test results are documented. Errors detected in this phase are communicated back to the development team for rework.

System Testing

The entire app is tested in an environment that is like the final execution environment for the app. Errors at this stage occur when various modules in the app do not interface correctly with one another. System testing is performed in a near Production environment. Load and performance testing are also part of this step. DevOps is involved in this step to help fix system performance related issues.

Acceptance Testing

The completed app is presented to the customer for Acceptance testing. Testing in this step may be conducted solely by the customer, or it can be performed jointly by the customer and the design team. Once this step has concluded to the satisfaction of the customer, the app is ready to be deployed.

EXPLAIN HOW AN APP IS MAINTAINED

In this section you will learn about deploying a software application and maintaining it.

Deployment

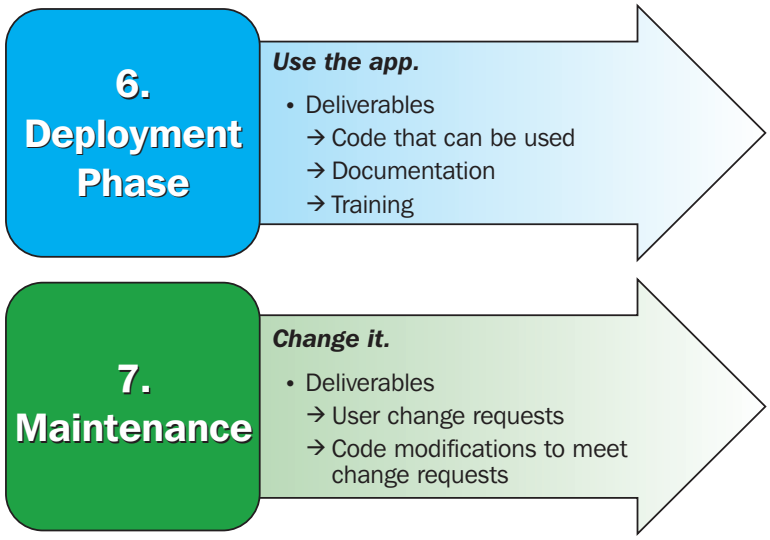


FIGURE 17. The Deployment and Maintenance Phases of the SDLC.

The last 2 phases of SDLC are the Deployment phase and the Maintenance phase. During the Deployment phase, code is considered operational and usable. Code is transitioned into its final, "real" environment which is referred to as the **Production Environment**. Transitioning code from the QA environment to the production environment is a complex task that used to be managed manually. Today, DevOps personnel play an important role in this phase by automating the transitioning process using tools and coding scripts. Customer training is also part of the Deployment phase. Personnel need to be trained on the new system. If bugs or flaws are detected by the customer during Deployment, processes need to be set up to report it, so that it can be fixed by the Development or Infrastructure team. Traditionally, the Development team steps away from the project during this phase and it is handled by Operations or DevOps. However, outcomes are better when Development, Design, DevOps and Operations work together in this phase.

Copyright © by CAERT, Inc. — Reproduction by subscription only.

Maintenance

In the maintenance phase, activities are performed to enhance the application. It lists the different categories of maintenance activities. These include code modifications and website monitoring activities.

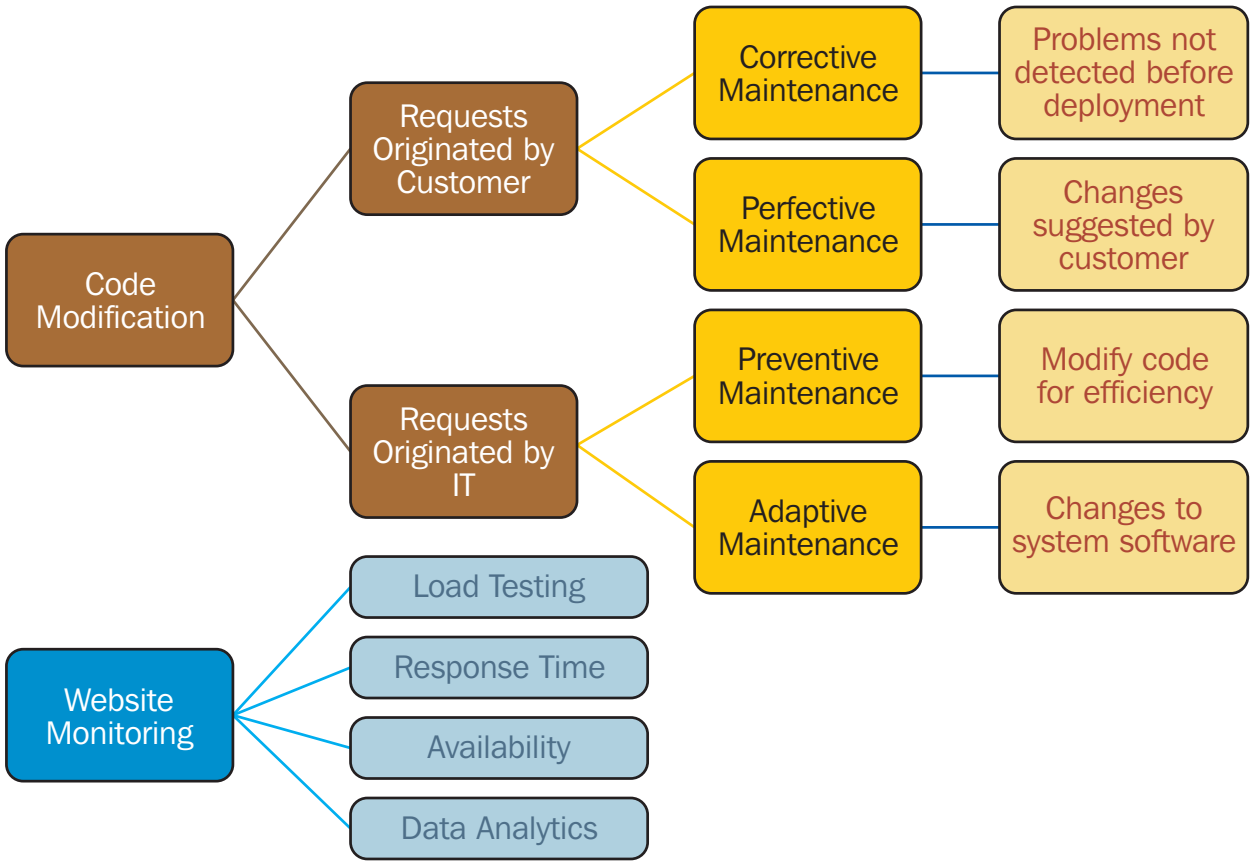


FIGURE 18. Types of Maintenance Activities.

Code Modification Activities

Code changes may be originated by the customer or IT after deployment.

- Customer Initiated Changes: The first type of customer originated change is Corrective Maintenance and it is performed due to flaws in the system that are identified by the customer. This may be the result of inaccurate application design or due to errors that surface after deployment. There needs to be a process to collect these errors from the customer and channel them to the appropriate team for fixes. The second type of change that originates from the customer is Perfective maintenance. These are enhancements or changes to the system that the Customer requests, which are not due to system flaws. Requirements may have changed from the beginning the SDLC and operational code may not meet new requirements. These new requirements are collected and change order processing is initiated to modify code.

Copyright © by CAERT, Inc. — Reproduction by subscription only.



FURTHER EXPLORATION...

ONLINE CONNECTION:
Exactly How Many Phases Are There in the SDLC?

This lesson discusses the 7 phases in the SDLC. However, there are also 5-phase and 6-phase SDLC models. Linkedin shows the SDLC as having 6 steps. The Planning phase is omitted. This is not to say that Planning is an unimportant task in a SDLC project! The 6-phase model incorporates our 7-phase model's first phase, the Planning phase, into the first phase of their SDLC which is called "Requirement gathering and analysis". The Coding phase is called the Implementation phase. The tasks performed in this phase are identical.

To learn more about Linkedin's SDLC discussion, go to https://www.linkedin.com/pulse/what-software-development-life-cycle-sdlc-phases-private-limited.

- IT initiated changes: The first type of change initiated by IT is Adaptive Maintenance. For example, an upgrade to the Operating System may require code to be changed. Changes to versions of Database Management Software may require changes to programs. Preventive Maintenance include tasks that are performed to enhance code without modifying its functionality. For example, code may be refactored to make it more efficient. Enhancements to documentation also fall under this category.

Website Monitoring

During the maintenance phase of a web app, the website is fully operational and handles normal activity. A true picture of request response time can be assessed at this stage since it is difficult to accurately predict response time in earlier stages of the SDLC. Software tools are used to measure website performance and third-party services are available as well. Besides monitoring performance, there is a movement towards the usage of Data analytics to assess website performance. Website analytics tools collect data, collate them and provide dashboards that display **Key Performance Indicators (KPIs)**, which are measures that the customer wishes to monitor, such as uptime, availability or response time.

Summary:



The Software Development Lifecycle (SDLC) is a model which is used in software application development projects. The SDLC consists of 7 phases where requirements are refined from high-level requirements to low-level coding details. There are many implementations of SDLC, and each implementation has its own advantages and disadvantages. This lesson discussed the tasks completed in each phase of the SDLC, along with the most popular SDLC implementations in use today.

Copyright © by CAERT, Inc. — Reproduction by subscription only.

Checking Your Knowledge:

1. What are the seven steps of the Software Development Lifecycle?
2. In which phase of the SDLC is code written and tested?
3. What is the difference between unit testing and system testing?
4. Why are programs changed after they have been deployed?
5. What are the advantages and disadvantages of the Agile Model?

Expanding Your Knowledge:

Use the knowledge in this E-unit to create a presentation explaining how the SDLC can be used to build an app that you wish to develop. Describe the tasks that will be completed in each phase in 2-3 paragraphs. Share your presentation with the class.

Web Links:

Introduction to the SDLC
https://www.youtube.com/watch?v=i-QyW8D3ei0&t=486s

Introduction to the Waterfall Model
https://www.youtube.com/watch?v=Y_A0E1ToC_I

Introduction to Scrum
https://www.agilelearninglabs.com/resources/scrum-introduction/

Copyright © by CAERT, Inc. — Reproduction by subscription only.